

Performance Testing

Date	04-07-2026
Team ID	
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. This section documents the testing approach, tools used, and results obtained for the OptiCrop Flask application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual testing (browser-based, Flask)
Type of Testing	Functional testing and performance testing
Target Module / API	Crop prediction module (/predict route, ML Model)
Test Environment	Local system (Windows, Flask dev server) and Render (production)
Test Date	02/07/2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Flask application (Home page)	1	10	Application loads successfully without errors
2	Submit valid crop input values on Find Your Crop page	1	20	Correct crop recommendation is displayed
3	Multiple prediction requests	5	30	Application responds correctly without crashing
4	Invalid/empty input validation	1	15	Error message or validation prompt is shown

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	1.1 sec	Pass	Response within target
2	Response Time (Max)	< 5 seconds	2.4 sec	Pass	Slightly higher on cold start (Render free tier)
3	Throughput (Req/sec)	10 Req/sec	12 Req/sec	Pass	Good request handling
4	Error Rate	< 1%	0%	Pass	No errors observed

5	CPU Utilization	< 80%	33%	Pass	CPU usage normal
6	Memory Utilization	< 80%	40%	Pass	Memory usage within limit

Step 4: Observations & Analysis

Key Findings:

The application responded quickly under normal usage. The Flask backend, StandardScaler transformation, and Logistic Regression prediction all executed with low response time and stable resource utilization across both local and Render-hosted environments.

Bottlenecks Identified:

No major bottlenecks were identified during normal operation. On Render's free tier, a cold-start delay of 30-50 seconds occurs after periods of inactivity, since the server spins down and needs to restart — this is a hosting-tier limitation, not an application performance issue.

Optimization Steps Taken:

The trained model and scaler are loaded once at application startup rather than on every request, avoiding repeated disk reads. Input processing uses efficient NumPy array operations, and Pandas is used only during training, not in the live prediction path, keeping the request-handling code lightweight.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below:

